Lab 16 – Database Design and Queries: Schema Design and SQL Generation

Hallticket no:2303A510A6

Batch no:02

Task 1 – Student Information System Schema

Task:

Design a database schema for a Student Information System and generate queries using AI.

Instructions:

• Tables: Students, Courses, Enrollments.

• Define primary keys, foreign keys, and relationships.

• Generate queries:

o Insert a new student record.

o Fetch all courses enrolled by a student.

o Count number of students in each course.

Expected Output:

• SQL CREATE TABLE statements and queries for student–course relationships.

Code:

mydatabase.sql

```sql
-- Create Students table
CREATE TABLE Students (
    student_id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL
);

-- Create Courses table
CREATE TABLE Courses (
    course_id INTEGER PRIMARY KEY AUTOINCREMENT,
    course_name TEXT NOT NULL
);

-- Create Enrollments table
CREATE TABLE Enrollments (
    enrollment_id INTEGER PRIMARY KEY AUTOINCREMENT,
    student_id INTEGER,
    course_id INTEGER,
    enrollment_date TEXT,
    FOREIGN KEY (student_id) REFERENCES Students(student_id),
    FOREIGN KEY (course_id) REFERENCES Courses(course_id)
);

-- Insert students
INSERT INTO Students (name) VALUES ('Alice');
INSERT INTO Students (name) VALUES ('Bob');
INSERT INTO Students (name) VALUES ('Charlie');

-- Insert courses
INSERT INTO Courses (course_name) VALUES ('Mathematics');
INSERT INTO Courses (course_name) VALUES ('Physics');
INSERT INTO Courses (course_name) VALUES ('Chemistry');

-- Insert enrollments
INSERT INTO Enrollments (student_id, course_id, enrollment_date)
VALUES (1,1,'2023-01');

INSERT INTO Enrollments (student_id, course_id, enrollment_date)
VALUES (2,2,'2023-01');

INSERT INTO Enrollments (student_id, course_id, enrollment_date)
VALUES (3,3,'2023-01');

--display all students
SELECT * FROM Students;

-- Insert a new student
INSERT INTO Students (name) VALUES ('David');

-- Fetch courses for a student
SELECT s.name, c.course_name, e.enrollment_date
FROM Students s
JOIN Enrollments e ON s.student_id = e.student_id
JOIN Courses c ON e.course_id = c.course_id
WHERE s.student_id = 1;
```

Output:

student_id	name
1	Alice
2	Bob
3	Charlie

SQL ▼ < 1 / 1

SQL ▼ < 1 / 1

name	course_name	enrollment_date
Alice	Mathematics	2023-01

SQL ▼ < 1 / 1

course_name	student_count
Chemistry	1
Mathematics	1
Physics	1

Task 2 – Hospital Management Database

Task:

Create schema and queries for a Hospital Management System.

Instructions:

• Tables: Doctors, Patients, Appointments.

• Use AI to define constraints (unique IDs, valid dates).

• Generate queries:

o List all appointments for a specific doctor.

o Retrieve patient history by patient ID.

o Count total patients treated by each doctor.

Expected Output:

• Normalized schema and SQL queries with joins.

Code:

mydatabase.sql

```sql
-- Create Doctor table
CREATE TABLE doctor (
    doctor_id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
    specialty TEXT NOT NULL,
    phone_number TEXT NOT NULL
);
-- Create Patient table
CREATE TABLE patient (
    patient_id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
    date_of_birth TEXT NOT NULL,
    phone_number TEXT NOT NULL
);
-- Create Appointment table
CREATE TABLE appointment (
    appointment_id INTEGER PRIMARY KEY AUTOINCREMENT,
    doctor_id INTEGER NOT NULL,
    patient_id INTEGER NOT NULL,
    appointment_date TEXT NOT NULL,
    reason TEXT NOT NULL,
    FOREIGN KEY (doctor_id) REFERENCES doctor(doctor_id),
    FOREIGN KEY (patient_id) REFERENCES patient(patient_id)
);
-- Insert doctors
INSERT INTO doctor (name, specialty, phone_number) VALUES
('John', 'Cardiology', '555-1234'),
('Jane', 'Neurology', '555-5678'),
('Bob', 'Pediatrics', '555-9012');
-- Insert patients
INSERT INTO patient (name, date_of_birth, phone_number) VALUES
('Alice', '1985-06-15', '555-3456'),
('Charlie', '1990-09-20', '555-7890'),
('Eve', '1988-03-10', '555-2345');
-- Insert appointments
INSERT INTO appointment (doctor_id, patient_id, appointment_date, reason) VALUES
(1, 1, '2023-10-10 10:00:00', 'Routine checkup'),
(2, 2, '2023-10-11 14:30:00', 'Headache consultation'),
(3, 3, '2023-10-12 09:15:00', 'Annual physical');
-- Display tables
SELECT * FROM doctor;
SELECT * FROM patient;
SELECT * FROM appointment;
-- List all appointments for a specific doctor
SELECT a.appointment_id, d.name AS doctor_name, p.name AS patient_name,
       a.appointment_date, a.reason
FROM appointment a
JOIN doctor d ON a.doctor_id = d.doctor_id
JOIN patient p ON a.patient_id = p.patient_id
WHERE d.doctor_id = 1;
-- Retrieve patient history by patient ID
SELECT a.appointment_id, d.name AS doctor_name,
       a.appointment_date, a.reason
FROM appointment a
JOIN doctor d ON a.doctor_id = d.doctor_id
```

Output:

doctor_id	name	specialty	phone_number
1	John	Cardiology	555-1234
2	Jane	Neurology	555-5678
3	Bob	Pediatrics	555-9012

SQL ▾ 1 / 1 1 - 3 of 3

patient_id	name	date_of_birth	phone_number
1	Alice	1985-06-15	555-3456
2	Charlie	1990-09-20	555-7890
3	Eve	1988-03-10	555-2345

SQL ▾ 1 / 1 1 - 3 of 3

appointment_id	doctor_id	patient_id	appointment_date	reason
1	1	1	2023-10-10 10:00:00	Routine checkup
2	2	2	2023-10-11 14:30:00	Headache consultation
3	3	3	2023-10-12 09:15:00	Annual physical

SQL ▾ 1 / 1 1 - 1 of 1

appointment_id	doctor_name	patient_name	appointment_date	reason
1	John	Alice	2023-10-10 10:00:00	Routine checkup

SQL ▾ 1 / 1 1 - 1 of 1

appointment_id	doctor_name	appointment_date	reason
1	John	2023-10-10 10:00:00	Routine checkup

SQL ▾ 1 / 1 1 - 3 of 3

doctor_name	total_patients
John	1
Jane	1
Bob	1

Task 3 – Library Database

Task:

Design schema for a Library Management System.

Instructions:

• Tables: Books, Members, Loans.

• Use AI to suggest indexing strategy for faster queries.

• Generate queries:

o Retrieve all books currently issued.

o Find overdue books (loan date > 30 days).

o Count number of books loaned by each member.

Expected Output:

• Schema + SQL queries demonstrating joins and conditions.

Code:

mydatabase.sql

```sql
CREATE TABLE Books (
    BookID INTEGER PRIMARY KEY AUTOINCREMENT,
    Title TEXT NOT NULL,
    Author TEXT NOT NULL,
    PublishedYear INTEGER,
    Genre TEXT
);
CREATE TABLE Members (
    MemberID INTEGER PRIMARY KEY AUTOINCREMENT,
    FirstName TEXT NOT NULL,
    LastName TEXT NOT NULL,
    Email TEXT UNIQUE NOT NULL,
    JoinDate TEXT
);
CREATE TABLE Loans (
    LoanID INTEGER PRIMARY KEY AUTOINCREMENT,
    BookID INTEGER,
    MemberID INTEGER,
    LoanDate TEXT,
    ReturnDate TEXT,
    FOREIGN KEY (BookID) REFERENCES Books(BookID),
    FOREIGN KEY (MemberID) REFERENCES Members(MemberID)
);
-- insert sample data into Books table
INSERT INTO Books (Title, Author, PublishedYear, Genre) VALUES
('The Great Gatsby', 'F. Scott Fitzgerald', 1925, 'Fiction'),
('To Kill a Mockingbird', 'Harper Lee', 1960, 'Fiction'),
('1984', 'George Orwell', 1949, 'Dystopian'),
('Moby Dick', 'Herman Melville', 1851, 'Adventure');
-- insert sample data into Members table
INSERT INTO Members (FirstName, LastName, Email, JoinDate) VALUES
('Alice', 'Brown', 'alice@email.com', '2023-01-10'),
('Bob', 'Smith', 'bob@email.com', '2023-02-15'),
('Charlie', 'Johnson', 'charlie@email.com', '2023-03-20');
-- insert sample data into Loans table
INSERT INTO Loans (BookID, MemberID, LoanDate, ReturnDate) VALUES
(1, 1, '2024-03-01', NULL),
(2, 2, '2024-02-10', NULL),
(3, 1, '2024-01-01', '2024-01-15');
-- display tables
SELECT * FROM Books;
SELECT * FROM Members;
SELECT * FROM Loans;
-- retrieve all books currently isued
SELECT b.Title, b.Author, m.FirstName, m.LastName, l.LoanDate
FROM Loans l
JOIN Books b ON l.BookID = b.BookID
JOIN Members m ON l.MemberID = m.MemberID
WHERE l.ReturnDate IS NULL;
-- find overdue books (loan date > 30 days)
SELECT b.Title, m.FirstName, m.LastName, l.LoanDate
FROM Loans l
JOIN Books b ON l.BookID = b.BookID
JOIN Members m ON l.MemberID = m.MemberID
WHERE julianday('now') - julianday(l.LoanDate) > 30
```

Output:

SQL ▾ < 1 / 1 > 1 - 4 of 4

BookID	Title	Author	PublishedYear	Genre
1	The Great Gatsby	F. Scott Fitzgerald	1925	Fiction
2	To Kill a Mockingbird	Harper Lee	1960	Fiction
3	1984	George Orwell	1949	Dystopian
4	Moby Dick	Herman Melville	1851	Adventure

SQL ▾ < 1 / 1 > 1 - 3 of 3

MemberID	FirstName	LastName	Email	JoinDate
1	Alice	Brown	alice@email.com	2023-01-10
2	Bob	Smith	bob@email.com	2023-02-15
3	Charlie	Johnson	charlie@email.com	2023-03-20

SQL ▾ < 1 / 1 > 1 - 3 of 3

LoanID	BookID	MemberID	LoanDate	ReturnDate
1	1	1	2024-03-01	NULL
2	2	2	2024-02-10	NULL
3	3	1	2024-01-01	2024-01-15

SQL ▾ < 1 / 1 > 1 - 2 of 2

Title	Author	FirstName	LastName	LoanDate
The Great Gatsby	F. Scott Fitzgerald	Alice	Brown	2024-03-01
To Kill a Mockingbird	Harper Lee	Bob	Smith	2024-02-10

SQL ▾ < 1 / 1 > 1 - 2 of 2

Title	FirstName	LastName	LoanDate
The Great Gatsby	Alice	Brown	2024-03-01
To Kill a Mockingbird	Bob	Smith	2024-02-10

SQL ▾ < 1 / 1 > 1 - 3 of 3

FirstName	LastName	BooksLoaned
Alice	Brown	2
Bob	Smith	1
Charlie	Johnson	0

Task 4 – Real-Time Application: Online Shopping Database

Scenario:

Design a database for an E-commerce platform.

Requirements:

• Tables: Users, Products, Orders, OrderDetails.

• Generate queries to:

o Retrieve all orders by a user.

o Find the most purchased product.

o Calculate total revenue in a given month.

• AI should also suggest normalization improvements and query optimization.

Expected Output:

• Complete schema with relationships + SQL queries for analytics.

Code:

mydatabase.sql

```sql
CREATE TABLE Users (
    UserID INTEGER PRIMARY KEY AUTOINCREMENT,
    Name TEXT NOT NULL,
    Email TEXT UNIQUE NOT NULL,
    JoinDate TEXT
);
CREATE TABLE Products (
    ProductID INTEGER PRIMARY KEY AUTOINCREMENT,
    ProductName TEXT NOT NULL,
    Price REAL NOT NULL,
    Stock INTEGER
);
CREATE TABLE Orders (
    OrderID INTEGER PRIMARY KEY AUTOINCREMENT,
    UserID INTEGER,
    OrderDate TEXT,
    FOREIGN KEY (UserID) REFERENCES Users(UserID)
);
CREATE TABLE OrderDetails (
    OrderDetailID INTEGER PRIMARY KEY AUTOINCREMENT,
    OrderID INTEGER,
    ProductID INTEGER,
    Quantity INTEGER,
    Price REAL,
    FOREIGN KEY (OrderID) REFERENCES Orders(OrderID),
    FOREIGN KEY (ProductID) REFERENCES Products(ProductID)
);
-- insert sample users
INSERT INTO Users (Name, Email, JoinDate) VALUES ('Alice', 'alice@email.com', '2024-01-10');
INSERT INTO Users (Name, Email, JoinDate) VALUES ('Bob', 'bob@email.com', '2024-02-12');
INSERT INTO Users (Name, Email, JoinDate) VALUES ('Charlie', 'charlie@email.com', '2024-03-05');
-- insert sample products
INSERT INTO Products (ProductName, Price, Stock) VALUES ('Laptop', 999.99, 10);
INSERT INTO Products (ProductName, Price, Stock) VALUES ('Smartphone', 499.99, 20);
INSERT INTO Products (ProductName, Price, Stock) VALUES ('Headphones', 199.99, 15);
-- insert sample orders
INSERT INTO Orders (UserID, OrderDate) VALUES (1, '2024-03-01');
INSERT INTO Orders (UserID, OrderDate) VALUES (2, '2024-03-02');
INSERT INTO Orders (UserID, OrderDate) VALUES (1, '2024-03-15');
-- insert sample order details
INSERT INTO OrderDetails (OrderID, ProductID, Quantity, Price) VALUES (1, 1, 1, 999.99);
INSERT INTO OrderDetails (OrderID, ProductID, Quantity, Price) VALUES (1, 3, 2, 199.99);
INSERT INTO OrderDetails (OrderID, ProductID, Quantity, Price) VALUES (2, 2, 1, 499.99);

INSERT INTO OrderDetails (OrderID, ProductID, Quantity, Price) VALUES (3, 1, 1, 999.99);

-- display tables
SELECT * FROM Users;
SELECT * FROM Products;
SELECT * FROM Orders;
SELECT * FROM OrderDetails;
-- retrieve all orders by a user
SELECT u.Name, o.OrderID, o.OrderDate
FROM Orders o
JOIN Users u ON o.UserID = u.UserID
```

SQL ▾ 1 / 1 1 - 3 of 3

UserID	Name	Email	JoinDate
1	Alice	alice@email.com	2024-01-10
2	Bob	bob@email.com	2024-02-12
3	Charlie	charlie@email.com	2024-03-05

SQL ▾ 1 / 1 1 - 3 of 3

ProductID	ProductName	Price	Stock
1	Laptop	999.99	10
2	Smartphone	499.99	20
3	Headphones	199.99	15

SQL ▾ 1 / 1 1 - 3 of 3

OrderID	UserID	OrderDate
1	1	2024-03-01
2	2	2024-03-02
3	1	2024-03-15

SQL ▾ 1 / 1 1 - 4 of 4

OrderDetailID	OrderID	ProductID	Quantity	Price
1	1	1	1	999.99
2	1	3	2	199.99
3	2	2	1	499.99
4	3	1	1	999.99

SQL ▾ 1 / 1 1 - 2 of 2

Name	OrderID	OrderDate
Alice	1	2024-03-01
Alice	3	2024-03-15

SQL ▾ 1 / 1 1 - 1 of 1

ProductName	TotalPurchased
Laptop	2

SQL ▾ 1 / 1 1 - 1 of 1

TotalRevenue
2899.95

Task 5 – University Examination Database

Design a database schema for a University Examination System and generate SQL queries using AI.

Instructions:

• Tables: Students, Subjects, Exams, Results.

• Define primary keys, foreign keys, and referential integrity constraints.

• Generate queries:

o Insert examination results for a student.

o Retrieve subject-wise marks for a student.

o Calculate average marks for each subject.

Expected Output:

• Normalized schema and SQL queries involving aggregation and Joins

Code:

```sql
CREATE TABLE Students (
    StudentID INTEGER PRIMARY KEY AUTOINCREMENT,
    Name TEXT NOT NULL,
    Email TEXT UNIQUE NOT NULL,
    Department TEXT
);
CREATE TABLE Subjects (
    SubjectID INTEGER PRIMARY KEY AUTOINCREMENT,
    SubjectName TEXT NOT NULL,
    Credits INTEGER
);
CREATE TABLE Exams (
    ExamID INTEGER PRIMARY KEY AUTOINCREMENT,
    SubjectID INTEGER,
    ExamDate TEXT,
    FOREIGN KEY (SubjectID) REFERENCES Subjects(SubjectID)
);
CREATE TABLE Results (
    ResultID INTEGER PRIMARY KEY AUTOINCREMENT,
    StudentID INTEGER,
    ExamID INTEGER,
    Marks INTEGER,
    FOREIGN KEY (StudentID) REFERENCES Students(StudentID),
    FOREIGN KEY (ExamID) REFERENCES Exams(ExamID)
);
-- insert sample students
INSERT INTO Students (Name, Email, Department) VALUES
('Alice', 'alice@univ.com', 'Computer Science'),
('Bob', 'bob@univ.com', 'Information Technology'),
('Charlie', 'charlie@univ.com', 'Electronics');
-- insert subjects
INSERT INTO Subjects (SubjectName, Credits) VALUES
('Database Systems', 4),
('Operating Systems', 4),
('Computer Networks', 3);
-- insert exams
INSERT INTO Exams (SubjectID, ExamDate) VALUES
(1, '2024-05-01'),
(2, '2024-05-03'),
(3, '2024-05-05');
-- insert examination results for students
INSERT INTO Results (StudentID, ExamID, Marks) VALUES
(1, 1, 85),
(1, 2, 78),
(2, 1, 72),
(2, 3, 80),
(3, 2, 90);
-- display tables
SELECT * FROM Students;
SELECT * FROM Subjects;
SELECT * FROM Exams;
SELECT * FROM Results;
-- retrieve subject-wise marks for a student
SELECT s.Name, sub.SubjectName, r.Marks
```

Python Debugger: Python File (ai assistes code)

```sql
-- display tables
SELECT * FROM Students;
SELECT * FROM Subjects;
SELECT * FROM Exams;
SELECT * FROM Results;
-- retrieve subject-wise marks for a student
SELECT s.Name, sub.SubjectName, r.Marks
FROM Results r
JOIN Students s ON r.StudentID = s.StudentID
JOIN Exams e ON r.ExamID = e.ExamID
JOIN Subjects sub ON e.SubjectID = sub.SubjectID
WHERE s.StudentID = 1;
-- calculate average marks for each subject
SELECT sub.SubjectName, AVG(r.Marks) AS AverageMarks
FROM Results r
JOIN Exams e ON r.ExamID = e.ExamID
JOIN Subjects sub ON e.SubjectID = sub.SubjectID
GROUP BY sub.SubjectName;
```

Output:

SQL ▼ 1 / 1 1 - 3 of 3

StudentID	Name	Email	Department
1	Alice	alice@univ.com	Computer Science
2	Bob	bob@univ.com	Information Technology
3	Charlie	charlie@univ.com	Electronics

SQL ▼ 1 / 1 1 - 3 of 3

SubjectID	SubjectName	Credits
1	Database Systems	4
2	Operating Systems	4
3	Computer Networks	3

SQL ▼ 1 / 1 1 - 3 of 3

ExamID	SubjectID	ExamDate
1	1	2024-05-01
2	2	2024-05-03
3	3	2024-05-05

SQL ▼ 1 / 1 1 - 5 of 5

ResultID	StudentID	ExamID	Marks
1	1	1	85
2	1	2	78
3	2	1	72
4	2	3	80
5	3	2	90

SQL ▼ 1 / 1 1 - 2 of 2

Name	SubjectName	Marks
Alice	Database Systems	85
Alice	Operating Systems	78

SQL ▼ 1 / 1 1 - 3 of 3

SubjectName	AverageMarks
Computer Networks	80.0
Database Systems	78.5
Operating Systems	84.0

Ln 72, Col 26 (1997 selected) Spaces: 4 UTF-8 CRLF { } SQLite